Chapter#05
Python Image Library (PIL)

Aj'Opas Wongtaweesap (OaTCoM)
E-mail : oatcomster@gmail.com

Agenda

- Introduction to Python Image Library
- PIL Download and Install Program
- Image Attributes and Methods
- Basic Digital Image Processing

Introduction to Python Image Library

- The **Python Imaging Library (PIL)** adds image processing capabilities to your Python interpreter.
- Pillow is the friendly PIL fork by Alex Clark and Contributors. PIL is the Python Imaging Library by Fredrik Lundh and Contributors.
- This library supports many file formats
- provides powerful image processing and graphics capabilities.
- The current free version is PILLOW 12.1.0

Python Image Library

- The core image library is designed for fast access to data stored in a few basic pixel formats.
- It should provide a solid foundation for a general image processing tool.
- Let's look at a few possible uses of this library:
  - Image Archives
  - Image Display
  - Image Processing

Image Archives

- The PIL is ideal for image archival and batch processing applications.
- can use the library to create thumbnails, convert between file formats, print images, etc.
- identifies and reads a large number of formats.
- Write support is intentionally restricted to the most commonly used interchange and presentation formats.

Image Display

- The current release includes **Tk *PhotoImage*** *and* ***BitmapImage interfaces***,
- *as well as a* Windows *DIB interface that can be used with PythonWin*.
- *For X and Mac displays, you can* use Jack Jansen's *img library*.
- For debugging, there's also a *show method in the Unix version which calls xv to display the* image.

Image Processing

- contains some basic image processing functionality, including point operations,
- filtering with a set of built-in convolution kernels, and color space conversions.
- supports image resizing, rotation and arbitrary affine transforms.
- There's a histogram method allowing you to pull some statistics out of an image.
- This can be used for automatic contrast enhancement, and for global statistical analysis.

Laboratory Workshop

- **WorkShop I** :: ให้นักศึกษาทำการติดตั้ง **PILLOW ver 12.1.0**
- ทดลองการใช้งานว่าสามารถเปิดใช้งาน Library ได้

PIL Importation

- PIL Importation by import Image for Python 3

>> from PIL import Image

- Create Image Objected by ***open*** method

```
>>> from PIL import Image
>>> im = Image.open("hopper.ppm")
```

- ***im*** is a Objected name for calling in Python programming

Attributes of Image

- **format** attribute identifies the source of an image. If the image was not read from a file, it is set to None
- **size** attribute is a 2-tuple containing width and height (in pixels)
- **mode** attribute defines the number and names of the bands in the image, and also the pixel type and depth. Common modes are "L" (luminance) for greyscale images, "RGB" for true colour images, and "CMYK" for pre-press images

Method Open, Show and Save

from PIL import Image
im = Image.open("filename")
im.show()

```
from PIL import Image
im = Image.open("Data.jpg")

im.show()
```

Attributes of Image

- **Example**

```
img = Image.open("test.jpg")
width, height = img.size
print(img.format, img.size, img.mode)
print(width, height)
```

- **Result**

```
>>>
PNG (311, 346) RGB
311 346
```

GetPixel and PutPixel

```
from PIL import Image
img = Image.open("Data.jpg")
r,g,b = img.getpixel((0,0))
print(r, g, b)
c = (r+g+b)//3
img.putpixel((0,0),c)
img.save("Data2.jpg")
```

RGB to Gray Scale Image

```
from PIL import Image
img = Image.open("Data.jpg")
w,h = img.size

for i in range(w):
  for j in range(h):
     r,g,b = img.getpixel((i,j))
     c = (r+g+b)//3
     img.putpixel((i,j),(c,c,c))

img.show()
img.save("DataGray.jpg")
```

นำค่าสีในแต่ละ pixel
แยกเป็นสี r,g,b
จากนั้นคำนวณหาค่าเฉลี่ย
แล้ว set ค่ากลับไปให้เท่ากัน
ทั้งค่าสี r,g,b

RGB to Gray Scale Image to B/W

```
from PIL import Image
img = Image.open("Data.jpg")
w,h = img.size

for i in range(w):
  for j in range(h):
     r,g,b = img.getpixel((i,j))
     c = (r+g+b)//3
     if(c>127): img.putpixel((i,j),(255,255,255))
     else: img.putpixel((i,j),(0,0,0))

img.show()
img.save("DataBW.jpg")
```

นำค่าสีเฉลี่ยค่า c มากำหนด
ว่าจะให้เป็นสีดำหรือสีขาว
เพราะภาพ B/W มี 2 สี
จึงกำหนดค่ากลาง 127 ขึ้นมา
สำหรับใช้เป็นตัวแบ่งสีดำ-สีขาว

Laboratory Workshop

- **WorkShop II** :: ให้นักศึกษาทำการเขียนโปรแกรมเพื่อทำการสลับค่าสีต่าง ๆ ตามโมเดลสีที่น่าสนใจ จากนั้นให้ทำการแปลงภาพสีเป็น Sepia

RGB to Sepia Image

```
from PIL import Image
img = Image.open("image.jpg")
w,h = img.size

for i in range(w):
  for j in range(h):
     r,g,b = img.getpixel((i,j))
     nr = int(0.393*r + 0.769*g + 0.189*b)
     ng = int(0.349*r + 0.686*g + 0.168*b)
     nb = int(0.272*r + 0.534*g + 0.131*b)

     if nr > 255: nr = 255
     else: nr = nr
     if ng > 255: ng = 255
     else: ng = ng
     if nb > 255: nb = 255
     else: nb = nb
     img.putpixel((i,j),(nr,ng,nb))

img.show()
```

ทำการดึงค่าสี r, g, b ของแต่ละ pixel ขึ้นมาก่อน จากนั้นคำนวณหาค่าสีใหม่ nr, ng, nb จากสมการ และค่าสีเป็นจำนวนเต็ม จึงจำเป็นต้องเปลี่ยนข้อมูลเป็น int จากนั้นค่าสีต้องอยู่ในช่วง 0-255 จึงต้องตรวจสอบเงื่อนไขสีทั้ง 3 ให้แน่ใจก่อนเซตกลับไป

Splitting and Merging bands

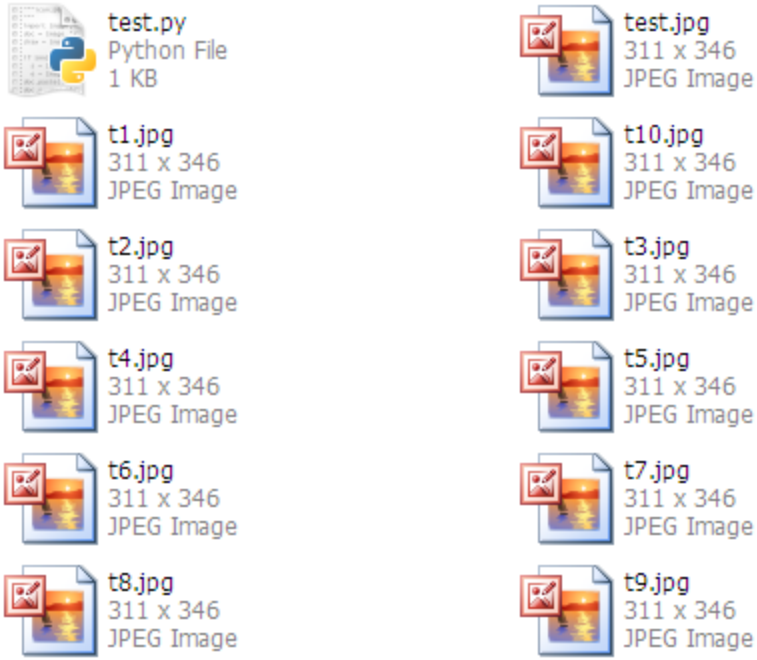
```
r, g, b = im.split()
im = Image.merge("RGB", (b, g, r))
```

- ทำการแยกสี r, g, b เก็บไว้ในตัวแปรเป็นแบบค่าแผ่นสี จากนั้นทำการรวมแผ่นสีทั้ง 3 ใหม่โดยในตัวอย่างเป็นการสลับค่าสีของแดงสลับกับสีฟ้า จะทำให้พื้นที่สีออกแดงส้มของภาพเดิมกลายเป็นสีฟ้าแทนที่

Open and Calculation

```
import Image
img = Image.open("test.jpg")
width, height = img.size
width  /= 2
height /= 2
print(width, height)
```

Image Duplication




```
import Image
img = Image.open("test.jpg")

for i in range(10):
    img.save("t" + str(i+1) + ".jpg")
```

Image Cropping

```
import Image
img = Image.open("test.jpg")
box = (10, 10, 150, 160)
newimg = img.crop(box)
newimg.show()
```

```
import Image
img = Image.open("test.jpg")
box = (0, 0, 500, 500)
newimg = img.crop(box)
newimg.show()
```

Image Transpose (Rotation)

```
import Image
img = Image.open("test.jpg")
img = img.transpose(Image.ROTATE_90)
img.show()
```

```
out = im.transpose(Image.FLIP_LEFT_RIGHT)
out = im.transpose(Image.FLIP_TOP_BOTTOM)
out = im.transpose(Image.ROTATE_90)
out = im.transpose(Image.ROTATE_180)
out = im.transpose(Image.ROTATE_270)
```

Image Rotation

```
import Image
img = Image.open("test.jpg")
img = img.rotate(20)
img.show()

img = Image.open("test.jpg").rotate(20).show()
```

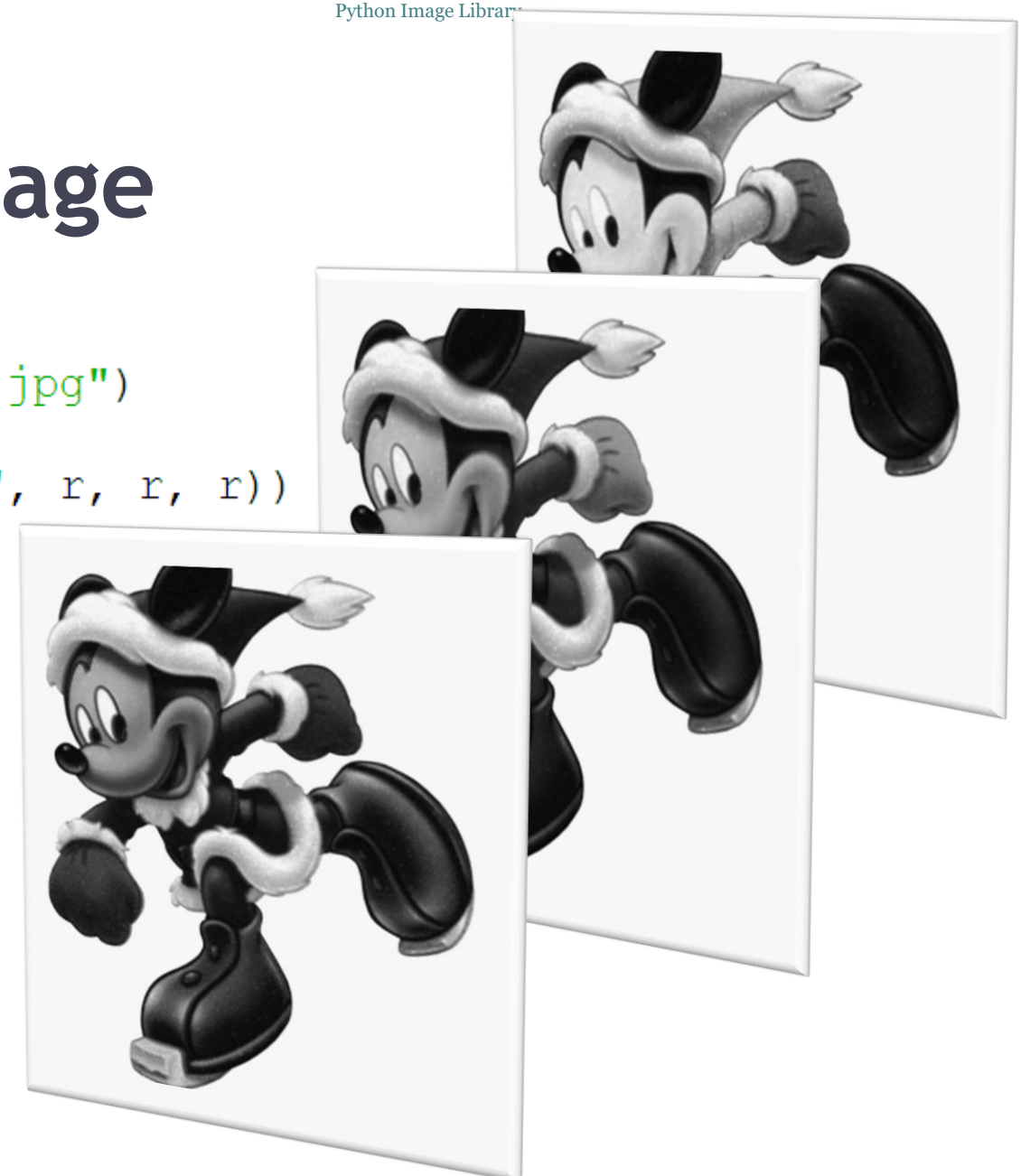
Resized Image

```
import Image
img = Image.open("test.jpg")
img = img.resize((200,200))
img.show()

import Image
img = Image.open("test.jpg").resize((200,200)).show()
```

Gray Scale Image

```
import Image
img = Image.open("test.jpg")
r, g, b = img.split()
img = Image.merge("RGB", r, r, r))
img.show()
```



Color Model Transformation

- **Mode** release supports the following standard modes:
  - 1 (1-bit pixels, black and white, stored as 8-bit pixels)
  - L (8-bit pixels, black and white)
  - P (8-bit pixels, mapped to any other mode using a colour palette)
  - RGB (3x8-bit pixels, true colour)
  - RGBA (4x8-bit pixels, true colour with transparency mask)
  - CMYK (4x8-bit pixels, colour separation)
  - YCbCr (3x8-bit pixels, colour video format)
  - I (32-bit integer pixels)
  - F (32-bit floating point pixels)

Color Model Transformation

```
import Image
img = Image.open("test.jpg")
img.convert("1").show()
img.convert("L").show()
```

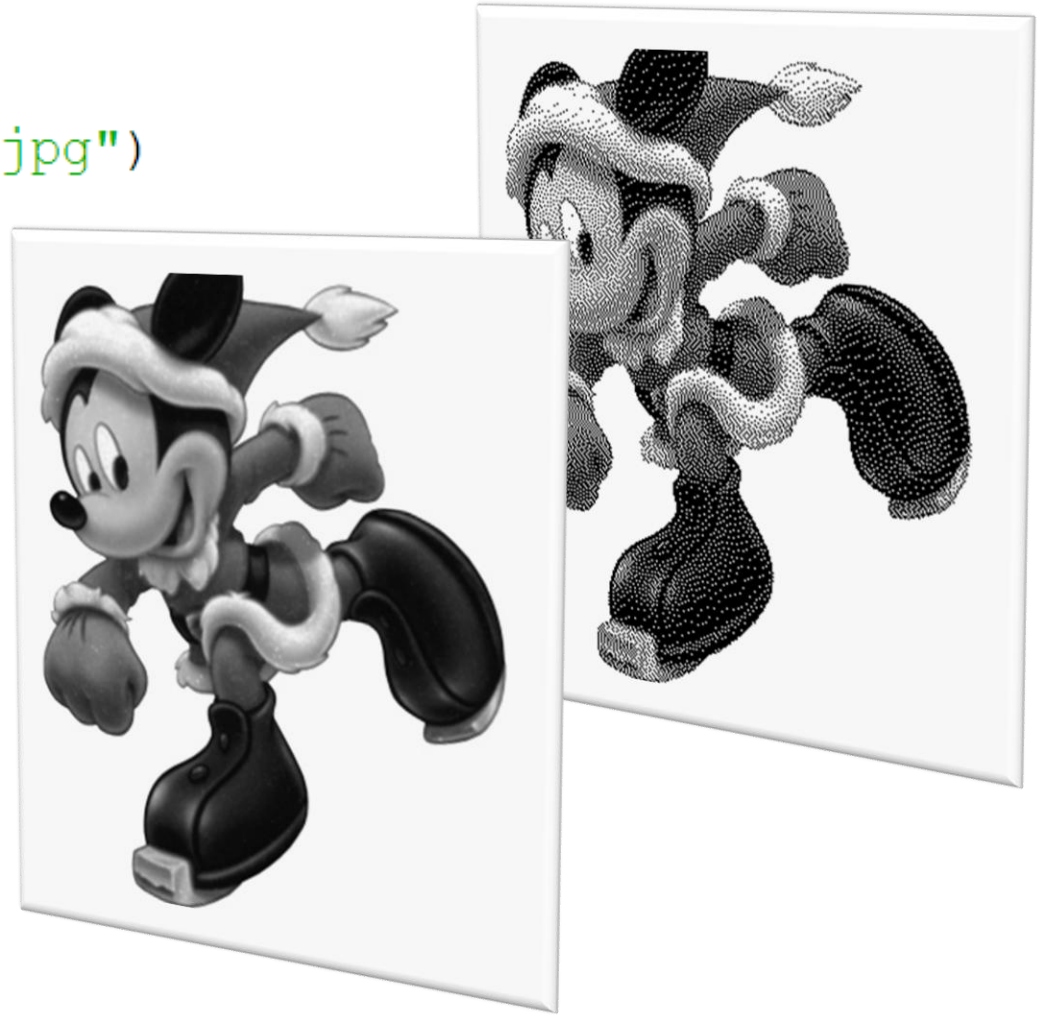


Image Histogram

```
import Image
img = Image.open("test.jpg")
c = img.histogram()
for i in range(768):
  print(i," == ",c[i])
```

Image Histogram

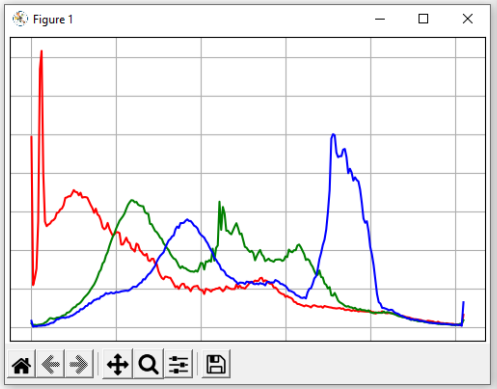
pil08-Histogram.py - C:/Users/Acer/AppData/Local/Programs/Python/Python310/pil08-Histogram.py (3.10.2)

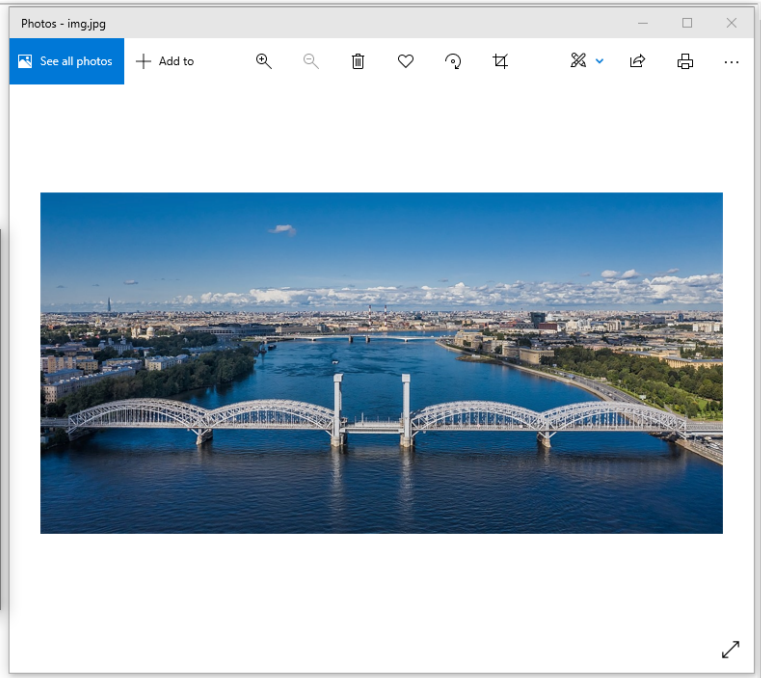
File Edit Format Run Options Window Help

```python
from PIL import Image
import matplotlib.pyplot as plt
import numpy as np

img = Image.open("img.jpg")
c = img.histogram()
nr = []
ng = []
nb = []
for i in range(256):
  nr.append(c[i])
  ng.append(c[i+256])
  nb.append(c[i+512])

plt.style.use('_mpl-gallery')
plt.xlabel('color value (0-255)')
plt.ylabel('pixel counts')
plt.plot(range(256),nr,c='r')
plt.plot(range(256),ng,c='g')
plt.plot(range(256),nb,c='b')
plt.show()
```





4 Sub-picture Splitation

```
from PIL import Image, ImageChops, ImageOps

image = Image.open("test.jpg")
xsize, ysize = image.size
a,b = xsize/2,ysize/2

part1 = image.crop((0, 0, a-1, b-1))
part2 = image.crop((a, 0, xsize, b-1))
part3 = image.crop((0, b, a-1, ysize))
part4 = image.crop((a, b, xsize, ysize))

#ImageOps.mirror(part4)
part1 = part1.rotate(180)

image1 = Image.new("RGB", image.size, (255,255,255))
image1.paste(part4, (a, b, xsize, ysize))
image1.paste(part1, (0, 0, a-1, b-1))

image1.show()
```

Invert Image

```
import Image, ImageChops
img = Image.open("test.jpg")
ImageChops.invert(img).show()
```

Image Mathematics

- Img1 + Img2
  - **ImageChops.add(image1, image2, scale, offset) => *image***
  - Adds two images, dividing the result by scale and adding the offset. If omitted, scale defaults to 1.0, and offset to 0.0.
  - out = (image1 + image2) / scale + offset
- Img1 – Img2
  - **ImageChops.difference(image1, image2) => *image***
  - Returns the absolute value of the difference between the two images.
  - out = abs(image1 - image2)

Example

```
from PIL import Image, ImageChops, ImageOps
img1 = Image.open("test.jpg")
img2 = Image.open("test.jpg")

img2 = ImageOps.mirror(img2)

ImageChops.add(img1, img2).show()
ImageChops.difference(img1, img1).show()
```

The ImageOps Module

- New in 1.1.3
- The ImageOps module contains a number of 'ready-made' image processing operations.
- This module is somewhat experimental, and most operators only work on L and RGB images.

ImageOps Methods

```
from PIL import Image, ImageOps
img = Image.open("test.jpg")
ImageOps.equalize(img).show()
ImageOps.flip(img).show()
ImageOps.grayscale(img).show()
ImageOps.invert(img).show()
ImageOps.mirror(img).show()
ImageOps.expand(img, 8, 0).show()
ImageOps.autocontrast(img, 0).show()
```

Laboratory Workshop

- **WorkShop III** :: ให้นักศึกษาทดลองเล่นเครื่องมือต่าง ๆ ในการประมวลผลภาพถ่ายจาก ImageOps

The ImageFilter Module

```
from PIL import Image, ImageFilter
img = Image.open("test.jpg")
img.filter(ImageFilter.BLUR).show()
```

Filters

- library provides the following set of predefined image enhancement filters:
- BLUR, CONTOUR, DETAIL, EDGE_ENHANCE, EDGE_ENHANCE_MORE, EMBOSS, FIND_EDGES, SMOOTH, SMOOTH_MORE, and SHARPEN.

Laboratory Workshop

- **WorkShop III** :: ให้นักศึกษาทดลองเล่นเครื่องมือต่าง ๆ ในการประมวลผลภาพถ่ายจาก Image Filter

Laboratory Workshop

- **WorkShop VI** :: ให้นักศึกษาเขียนโปรแกรมเพื่อประมวลผลภาพจากตัวอย่างที่ให้มาเพื่อให้พร้อมใช้งานกับการประมวลผลอื่น ๆ ต่อไป
- โจทย์ทดสอบ ให้นักศึกษาทำการแปลงภาพขาวดำให้อยู่ในสภาวะมองเห็นและอ่านได้ตามปกติ รวมทั้งทำการแปลงให้กลายเป็นภาพสี RGB อย่างสวยงาม ไฟล์ภาพชื่อ DataBW.jpg